

EECS 482

Introduction to operating systems

Deadlock prevention; address spaces

Peter Chen
University of Michigan

What to do about deadlocks

- Ignore
- Detect and fix
- Prevent
 - Think about what conditions are necessary for deadlock to occur

Four necessary conditions for deadlock

- **Limited resource**
 - Not enough to serve all threads simultaneously
 - Unlimited resource → no waiting for that resource
- **No preemption**
 - Can't force threads to give up resources
 - Preemption → no indefinite waiting
- **Hold and wait**
 - Hold resources while waiting to acquire others
 - No hold and wait → no multi-edge path in wait-for graph
- **Cycle of hold-wait requests**
 - No cyclical chain → no cycle in wait-for graph

How to eliminate limited resources?

- Increase # of resources
 - E.g., buy more machines
 - E.g., increase # of locks?

How to eliminate no preemption?

- Enable preemption
 - Preempt CPU
 - Preempt memory (next lecture topic)
 - Preempt lock?

Four necessary conditions for deadlock

- **Limited resource**
 - Not enough to serve all threads simultaneously
 - Unlimited resource → no waiting for that resource
- **No preemption**
 - Can't force threads to give up resources
 - Preemption → no indefinite waiting
- **Hold and wait**
 - Hold resources while waiting to acquire others
 - No hold and wait → no multi-edge path in wait-for graph
- **Cycle of hold-wait requests**
 - No cyclical chain → no cycle in wait-for graph

How to eliminate hold and wait?

- First step:
 - Move resource acquisition to beginning

```
phase 1          all needed
                ×
while (!done) {
    acquire some resource
    work
}
```

```
phase 2
    release all resources
```

How to eliminate hold and wait?

- Second step
 - Option A: Acquire all needed resources **atomically**

phase 1

```
acquire all needed resources atomic  
while (!done) {  
    work  
}
```

phase 2

```
release all resources
```


Atomic acquisition

`L.lock()`

`while right chopstick busy or left chopstick busy`

`cv.wait (L)`

`pick up right chopstick`

`pick up left chopstick`

`L.unlock()`

`<eat>`

`L.lock()`

`drop left chopstick`

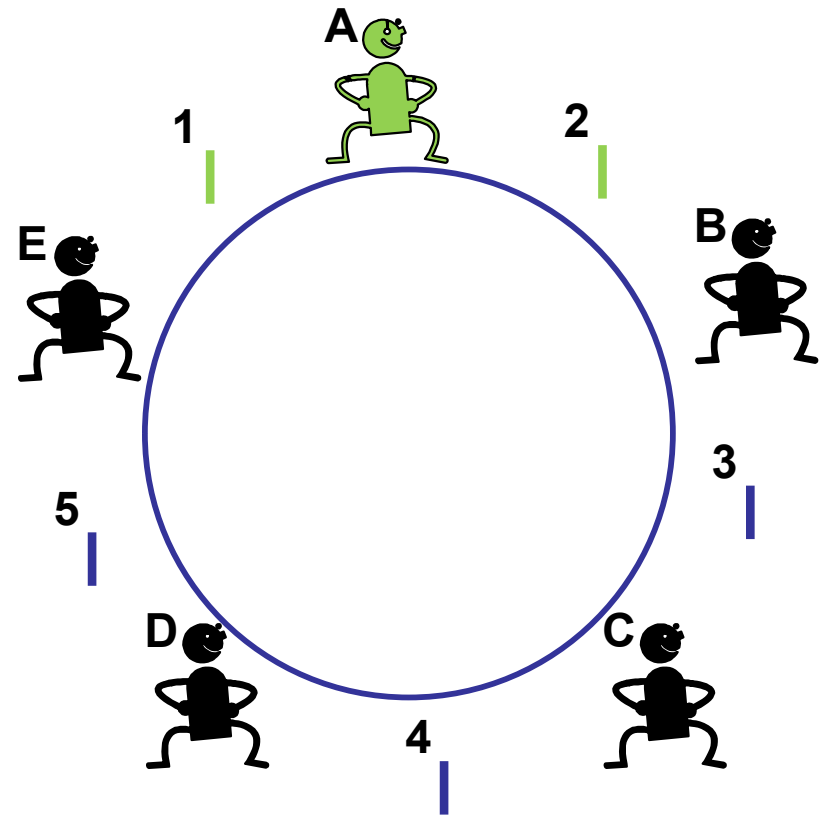
`drop right chopstick`

`cv.broadcast()`

`L.unlock()`

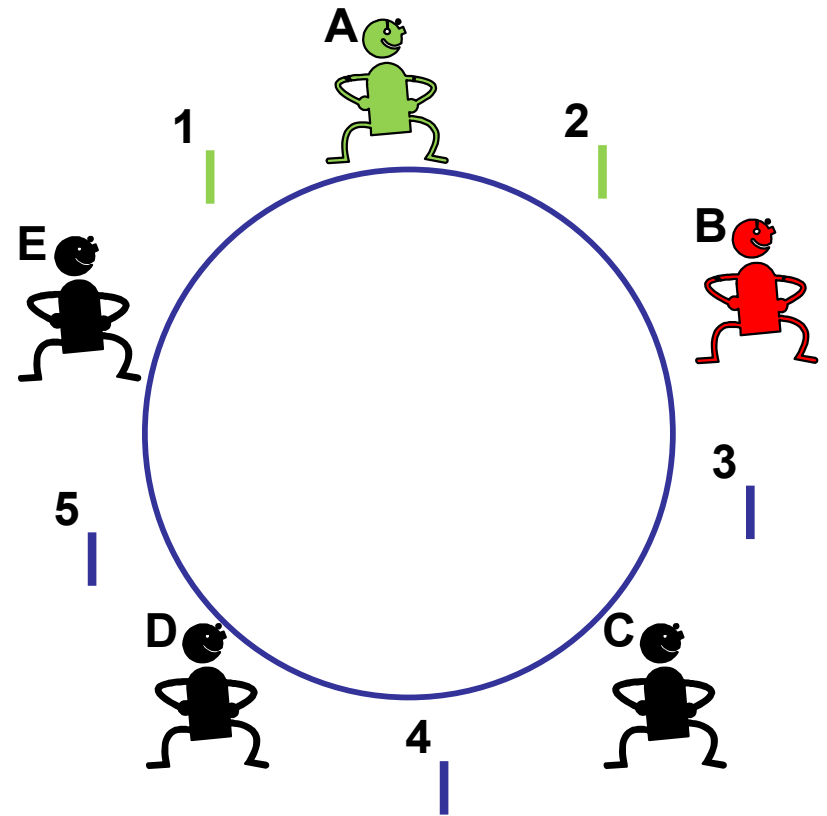
Dining philosophers

- A starts to eat



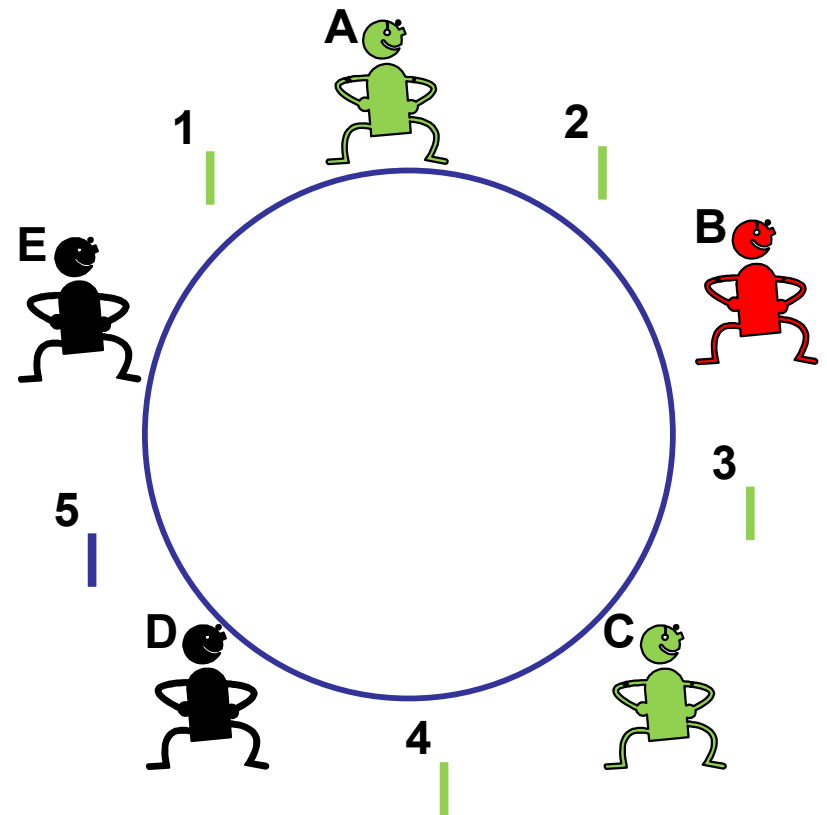
Dining philosophers

- A starts to eat
- B waits



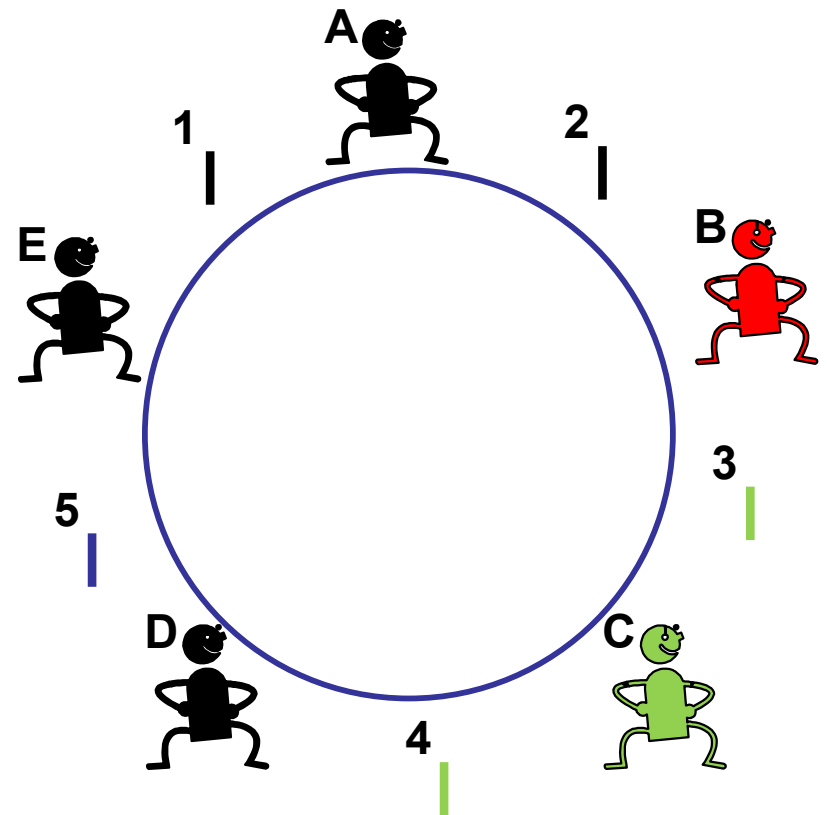
Dining philosophers

- A starts to eat
- B waits
- C starts to eat



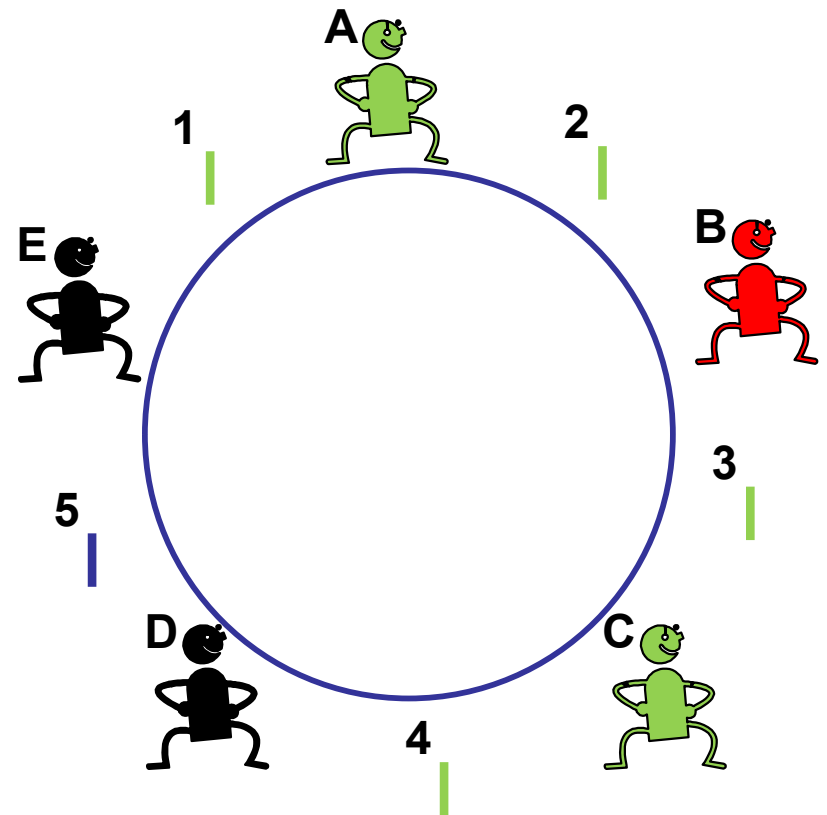
Dining philosophers

- A starts to eat
- B waits
- C starts to eat
- A finishes



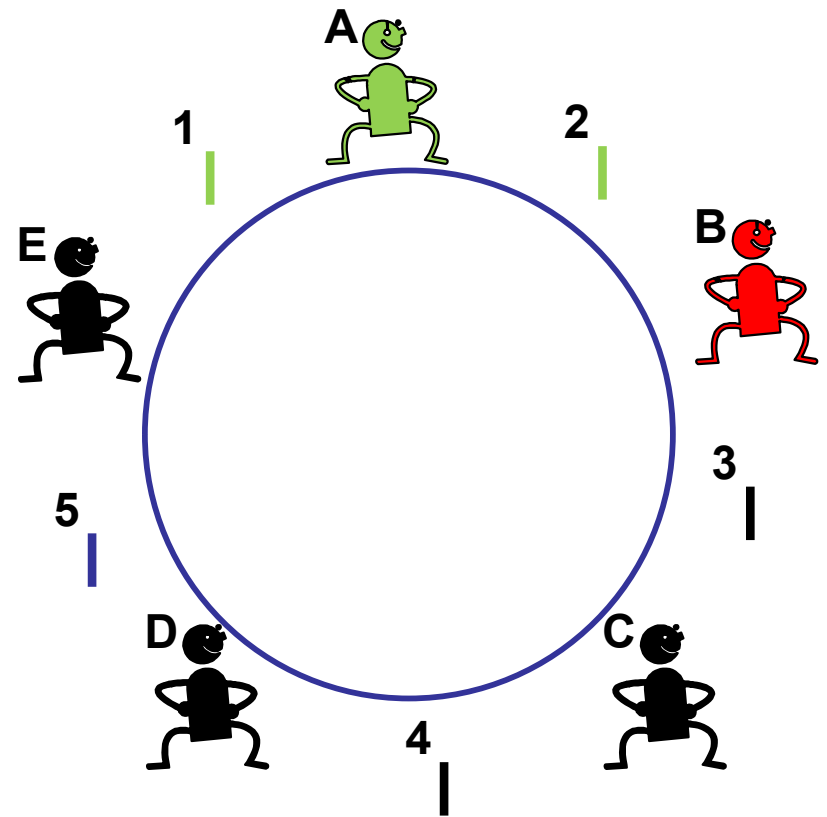
Dining philosophers

- A starts to eat
- B waits
- C starts to eat
- A finishes
- A starts to eat



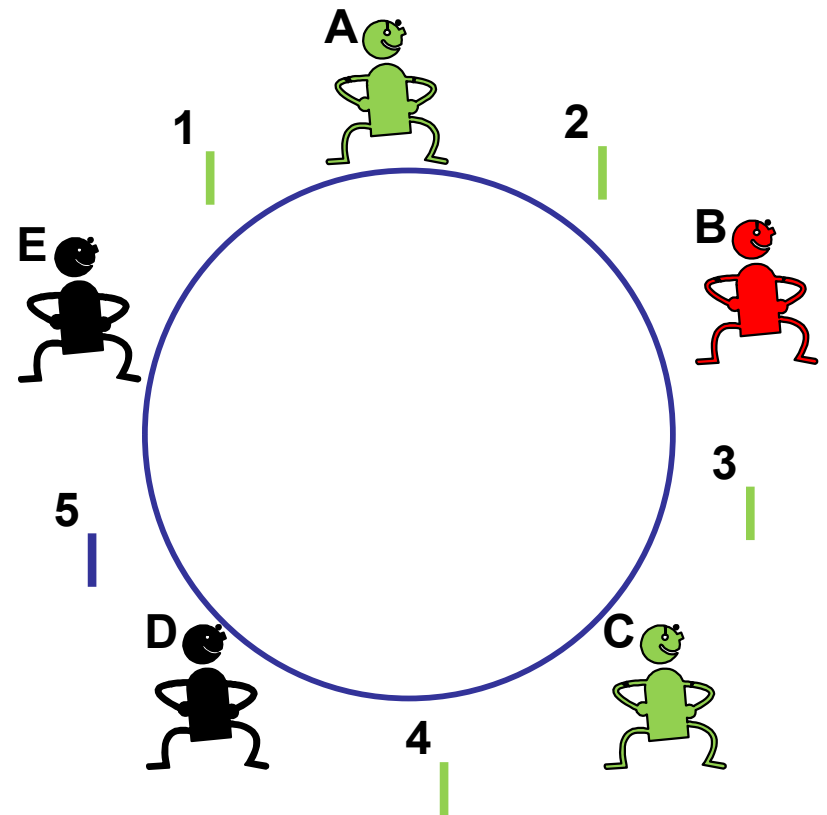
Dining philosophers

- A starts to eat
- B waits
- C starts to eat
- A finishes
- A starts to eat
- C finishes



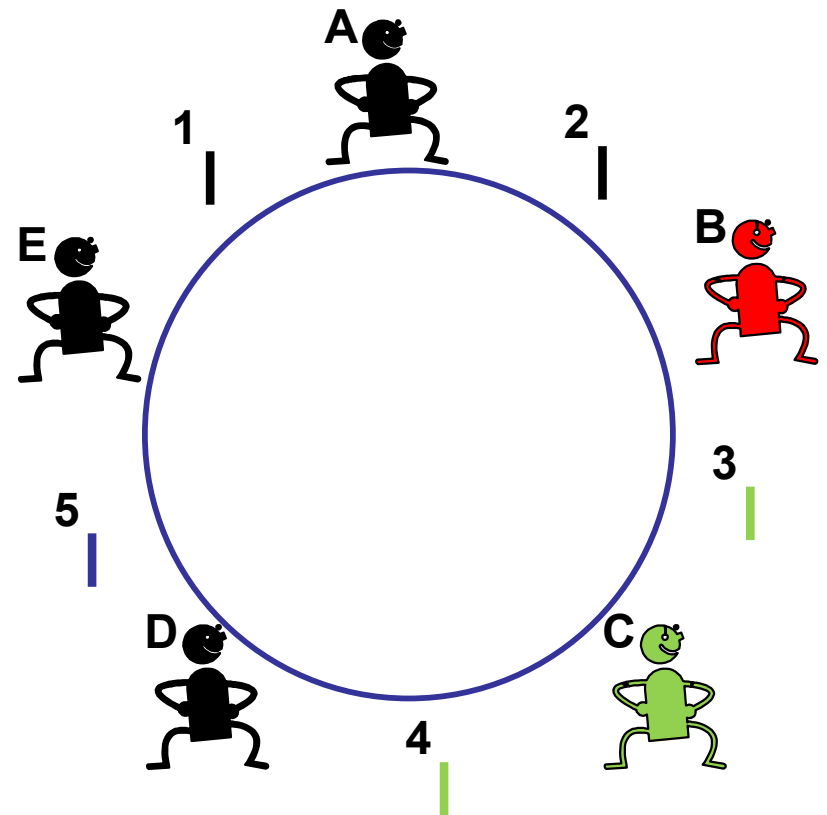
Dining philosophers

- A starts to eat
- B waits
- C starts to eat
- A finishes
- A starts to eat
- C finishes
- C starts to eat



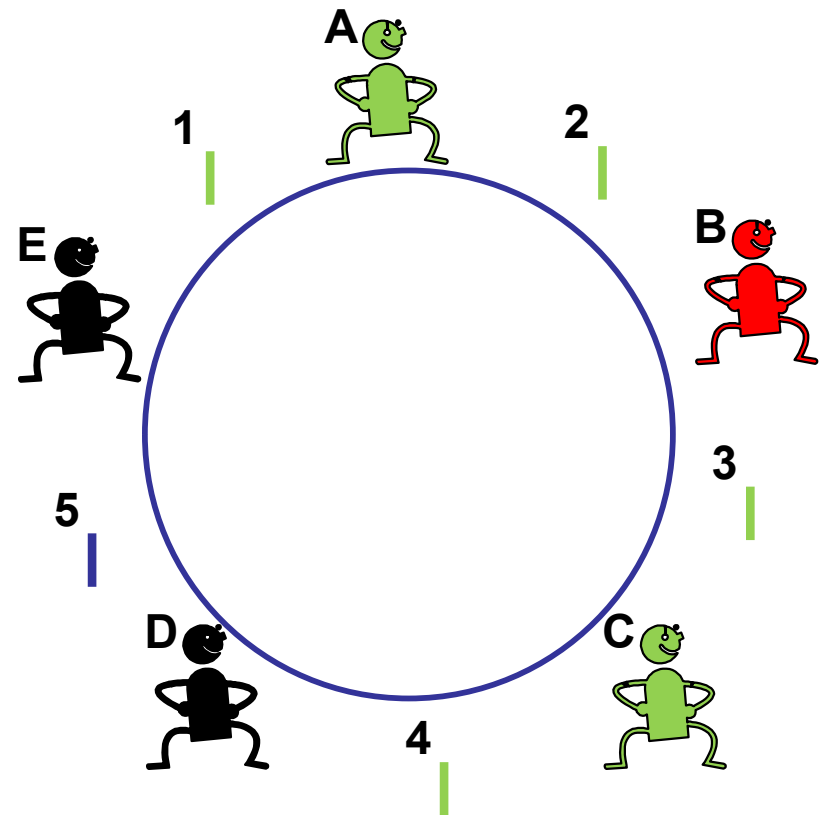
Dining philosophers

- A starts to eat
- B waits
- C starts to eat
- A finishes
- A starts to eat
- C finishes
- C starts to eat
- A finishes



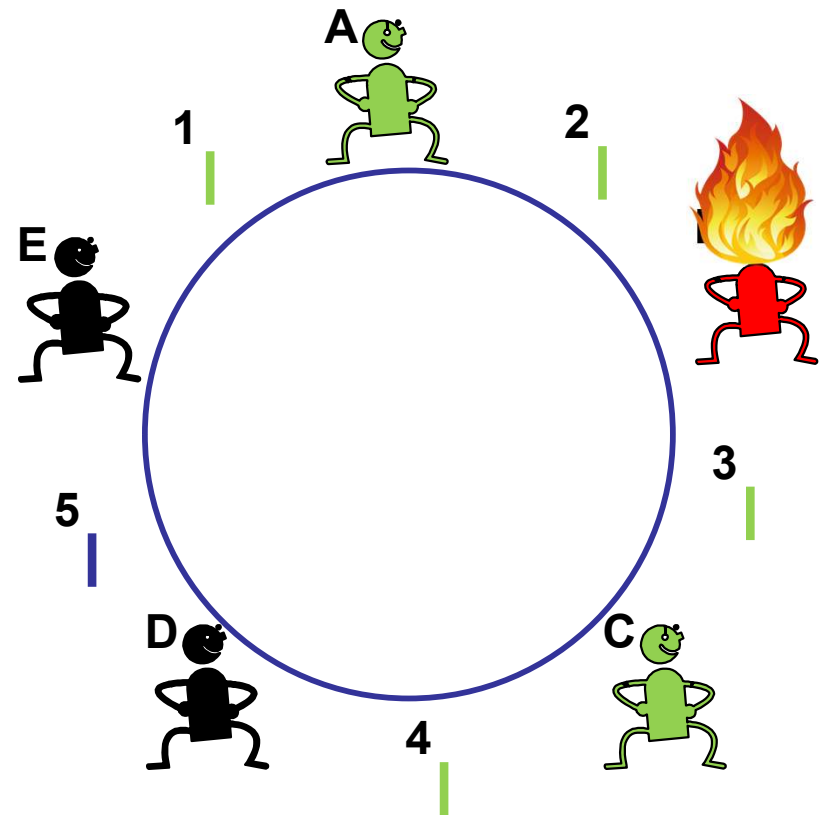
Dining philosophers

- A starts to eat
- B waits
- C starts to eat
- A finishes
- A starts to eat
- C finishes
- C starts to eat
- A finishes
- A starts to eat



Dining philosophers

- A starts to eat
- B waits
- while (1)
 - C starts to eat
 - A finishes
 - A starts to eat
 - C finishes



How to eliminate hold and wait?

- Second step
 - Option A: Acquire all needed resources **atomically**
 - Option B: Release and retry if you encounter a busy resource

phase 1

```
acquire all needed resources
while (!done) {
    work
}
```

phase 2

```
release all resources
```

How to eliminate hold and wait?

- What's the problem with this style of solution?

phase 1

```
acquire all needed resources
while (!done) {
    work
}
```

phase 2

```
release all resources
```

How to eliminate cycle of hold-and-wait?

- Define a global order over all resources
 - All threads **must** follow this order when acquiring resources
 - Guarantees that some thread can always make progress

Thread A

x.lock()

y.lock()

...

y.unlock()

x.unlock()

Thread B

y.lock()

x.lock()

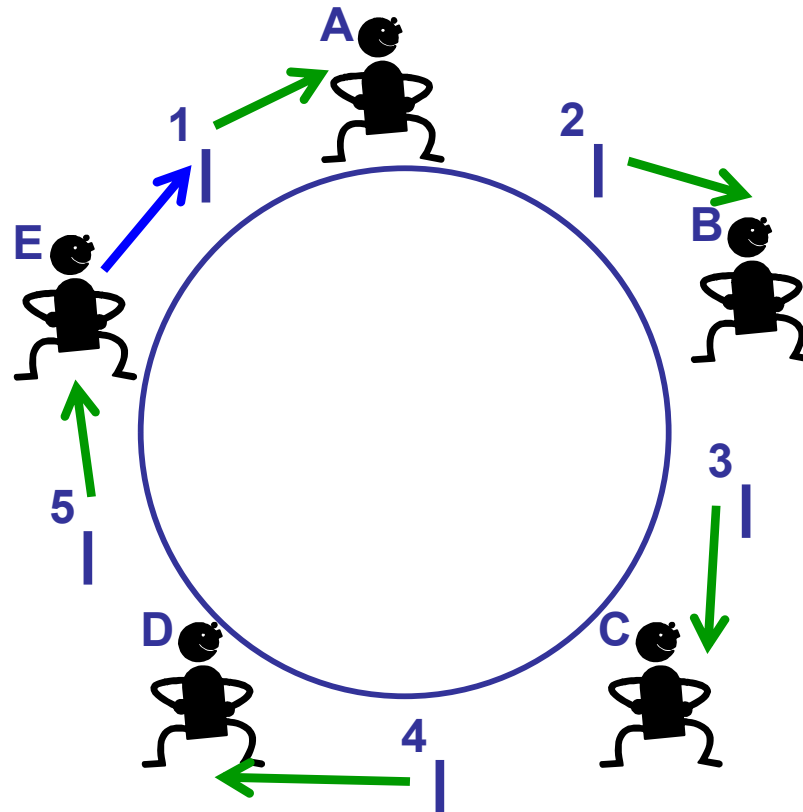
...

x.unlock()

y.unlock()

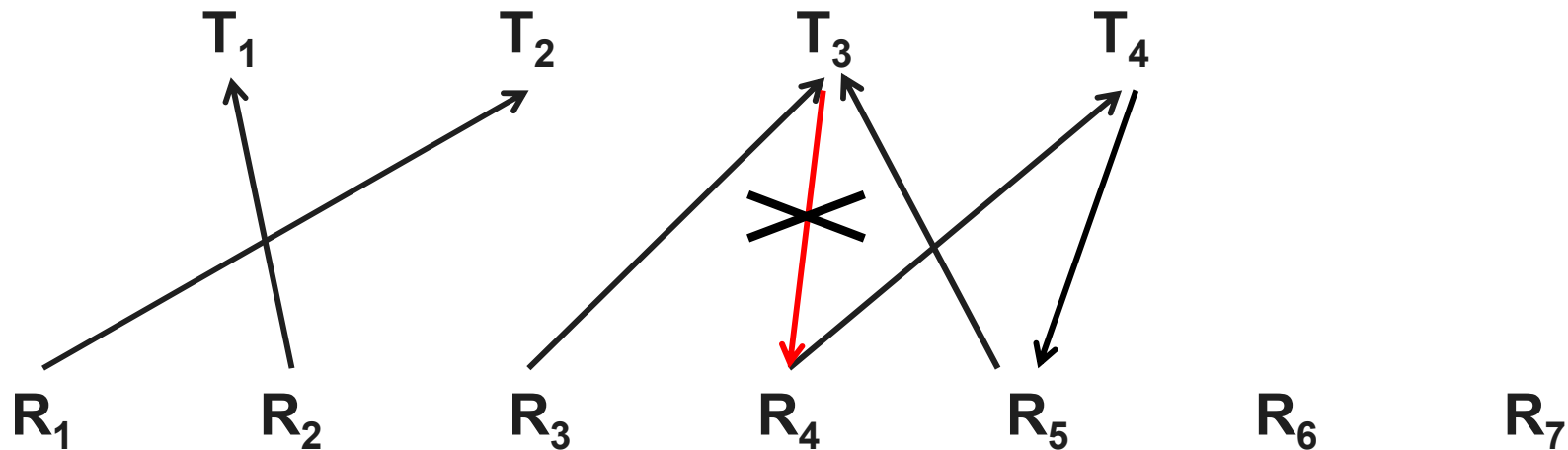
Dining philosophers

- Pick up lower # chopstick first
- Pick up higher # chopstick second



Global ordering of resources

- How can we be sure that *some* thread can make progress?



- Which thread can make progress?

Project 2: writing good test cases

- A good test suite provides comprehensive coverage
- Coverage along what dimension?
 - Code coverage
 - Specification coverage

Example: test if lock() blocks when mutex is held

Thread A

create thread B

m.lock()

m.unlock()

Thread B

m.lock()

m.unlock()

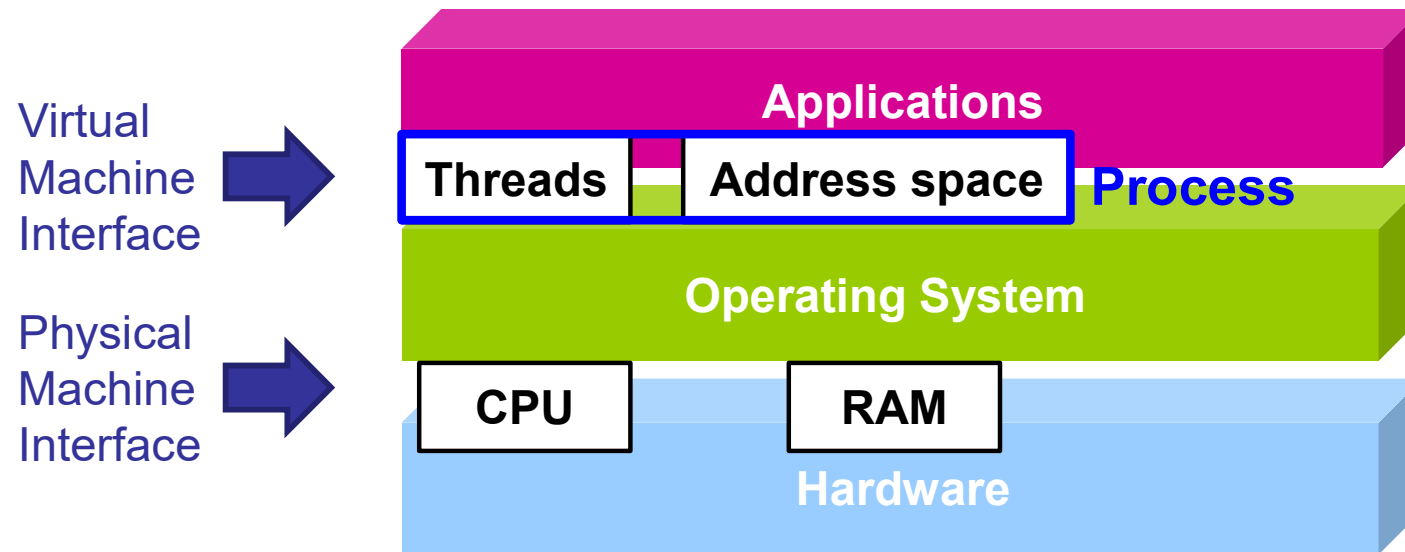
Writing good test cases

- Specification coverage
 - Targeted test cases (micro test case)
 - Stress test (macro test case)
- Which is better for debugging?

Midterm exam

- Oct. 20 @ 5:00-6:50 pm
- Topics
 - All lecture topics until end of deadlock
 - All labs on these topics
 - Projects 1 and 2
- Logistics
 - Closed book
 - Your room assignment will be posted on the web page
 - You will be entering your answers via a web form
 - Bring a laptop whose battery will last 2 hours
- Sample exams (<https://os.eecs.umich.edu/lab>)

OS abstractions



- Process = one or more threads in an address space

Address space

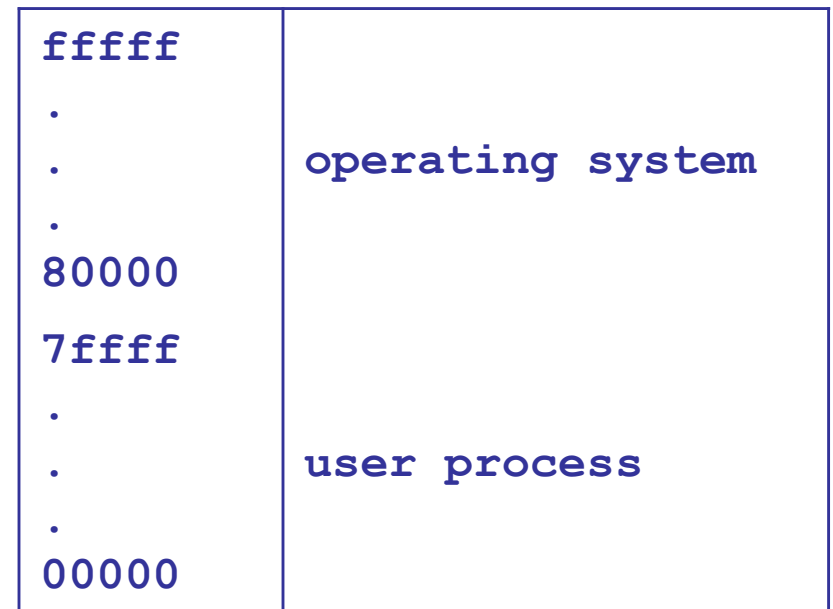
- All the memory space the process can use as it runs
 - Code
 - Stack
 - Data segment
- Physical machine interface: single memory shared by all jobs
- Virtual machine interface: each process has its own memory

Address space abstractions

- **Address independence**: same numeric address can be used in different address spaces, yet refer to distinct data items
- **Protection (controlled sharing)**: one process can't access data in another process's address space
- **Large address space**: address space for a process can be larger than physical memory

Uni-programming

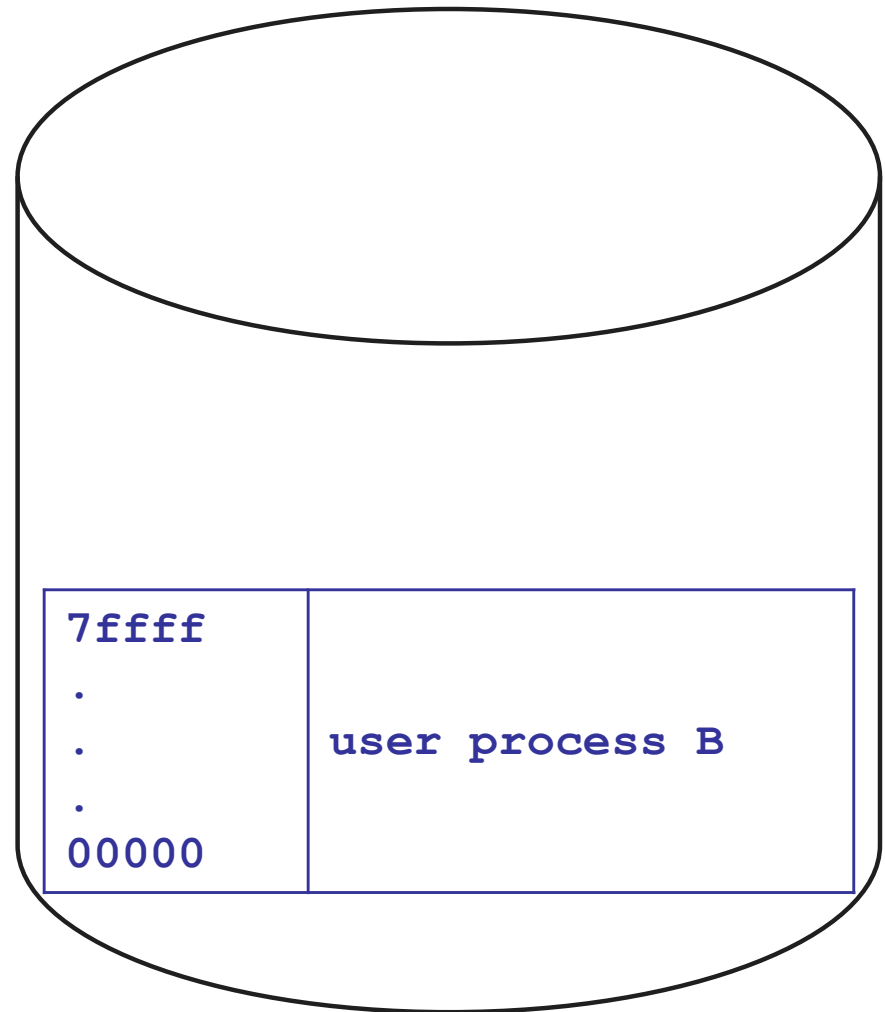
- 1 process occupies memory at a time
- Always load process into same spot in memory
- Reserve space for OS
- Virtual address (process view) = physical address (hardware view)



How to switch processes with uni-programming?

ffffff . . . 80000	operating system
7ffff . . . 00000	user process A

Physical memory



Disk

Uni-programming summary

- Address space abstractions
 - ✓ Address independence?
 - ✓ Protection?
 - ✗ Large address space?
- + Simple (early operating systems used this)
- Expensive context switch

Multi-programming

- Allow **multiple** processes to reside in physical memory at the same time
 - Makes it harder to provide protection
 - Makes it harder to provide address independence
- Providing protection and address independence requires system to do some work on **each memory access (!)**

Dynamic address translation



- **Address independence**
 - MMU translates same virtual address in different processes to different physical addresses
- **Protection**
 - MMU ensures that processes don't access each other's data
- **Large address space**
 - Virtual address only needs to be in physical memory when accessed